# Mini Project: Pricing a European Call Option in the Binomial Model

MTH790U/P Programming in C++ for Finance 2026/27

Lennart Dabelow
Queen Mary University of London

- Background: The binomial model
  - Stock prices
  - No arbitrage and the risk-neutral probability
  - Pricing a European call option
- Part 1: Setting up the project
- Part 2: The pricing model
- Part 3: The user interface
- Part 4: Documentation
- Part 5: Experiments and report
- Submission
- Checklist
- Feedback

---

In this mini project, you will bring together everything you have learned in Lectures 1–6 and write a complete small program from scratch. It computes the fair price of a European call option in the binomial model. You will give the program a simple menu-based user interface, document it, track its development with git, and use it to explore how the option price depends on the model parameters. You will summarize your findings in a short report.

**At a glance:**

- **Format:** Individual work. The project is **not assessed**; it is a chance to practise and to get feedback. As part of your assessed final project, you will be required to reflect on the feedback received on the mini project.
- **Deadline: Friday, 6 November 2026, 23:59 UK time, via QMPlus** (see “Submission” below).
- **Timing:** Start right after the Week 6 lecture. There are no classes in Week 7, so that you can work on your project.
- **What you submit:** your report (PDF, about two pages) and a zip of your complete project folder, including its git history.
- **Feedback:** In Week 8, you will test and review another student’s project and give them feedback (and receive feedback on yours). After that, you will receive written feedback from me.

**Allowed concepts: Only use the concepts and techniques that we have covered in Lectures 1–6.** Everything you need has been covered, and many building blocks already appeared in Problem Sets 1–5. Feel free to reuse (and improve) your own solutions. In particular, *don't* use structs, classes, exceptions, or other language and library features that we haven't discussed yet.

**Use of AI tools:** You may use AI tools. However, recall the discussion from Lecture 1: You are responsible for understanding, testing, and being able to explain every line of code you submit. AI tools will often suggest code that uses concepts we haven't covered in class. It is your job to detect this and change the code accordingly. In your report, state briefly whether and how you used AI tools.

# Background: The binomial model

The binomial model is the simplest model of a stock price that moves randomly over time, and it is widely used to price options. Time is divided into $N$ periods (e.g., days or months). There is a **stock** with current price $S_0$ and a **risk-free asset** (a bank account) that pays the interest rate $R$ per period. In each period, the stock price either goes **up** by the rate $U$ or **down** by the rate $D$. That is, it is multiplied by either $1 + U$ or $1 + D$. We use the following parameters:

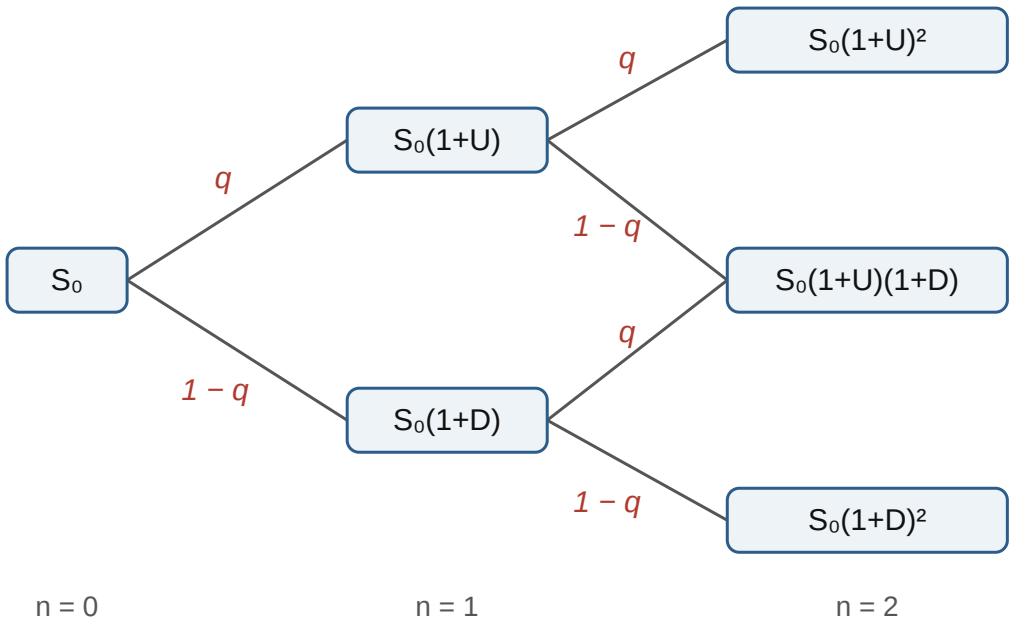
| Symbol | Meaning | Constraint | Default value |
|---|---|---|---|
| $S_0$ | current stock price | $S_0 > 0$ | 100.0 |
| $K$ | strike price of the option | $K \geq 0$ | 100.0 |
| $U$ | up rate per period | $U > -1$ | 0.05 |
| $D$ | down rate per period | $-1 < D$ | −0.05 |
| $R$ | risk-free interest rate per period | $R > -1$ | 0.01 |
| $N$ | number of periods | positive integer | 10 |

## Stock prices

After $n$ periods, of which $i$ were up-moves and $n - i$ were down-moves, the stock price is

$$S(n, i) = S_0 \, (1 + U)^i \, (1 + D)^{n-i}, \qquad i = 0, 1, \ldots, n.$$

Note that the order of the moves does not matter: up-then-down leads to the same price as down-then-up. The possible prices therefore form a *recombining tree*, shown here for two periods:




The labels $q$ and $1-q$ on the branches refer to the risk-neutral probabilities, which we introduce next.

## No arbitrage and the risk-neutral probability

The model only makes sense from a financial point of view if

$$D < R < U.$$

If this condition is violated, there is an **arbitrage**, i.e., a way to make a risk-free profit. For example, if $R \geq U$, you could sell the stock short and invest the proceeds in the bank account: Whatever the stock does, the bank account grows at least as fast. We call the model **arbitrage-free** if $D < R < U$.

In an arbitrage-free model, the number

$$q = \frac{R - D}{U - D}$$

lies strictly between 0 and 1. It is called the **risk-neutral probability** (of an up-move). If the stock went up with probability $q$, its expected return would equal the risk-free rate: $q(1+U) + (1-q)(1+D) = 1 + R$.

## Pricing a European call option

A **European call option** with strike $K$ and expiry $N$ gives its holder the right to buy the stock for the price $K$ at time $N$. Its payoff at time $N$ is

$$H(N, i) = \max\big(S(N, i) - K, 0\big),$$

i.e., exactly the `callPayoff` function from Problem Set 5. Here, $i$ denotes the number of up-moves up to time $N$, as before. The fair (no-arbitrage) price of the option at time 0 can be computed by working backwards through the tree: The value of the option at any node is the **expected value of the option one period later, computed with the risk-neutral probability $q$, discounted by one period**:

$$H(n, i) = \frac{q\, H(n+1, i+1) + (1-q)\, H(n+1, i)}{1 + R}, \qquad i = 0, 1, \ldots, n$$

Starting from the payoffs at time $N$, you apply this formula for $n = N - 1, N - 2, \ldots, 0$. The **price of the option** is the value $H(0, 0)$ at the root of the tree. This procedure is called **backward induction**. You don't need to derive or justify it in this project. If you are interested in the reasoning behind it, any textbook on mathematical finance will explain it (e.g., M. Capiński and T. Zastawniak, *Mathematics for Finance*).

**Hint:** You don't need to store the whole tree. A single `std::vector<double>` of length $N + 1$ is enough: It initially holds the payoffs at time $N$, and you overwrite its entries step by step. Think carefully about the order in which you update the entries. However, if you use a different data structure or storage layout, that's also fine if you explain it briefly in your report.

There is also an explicit formula for the price, which you can use to test your program:

$$H(0, 0) = \frac{1}{(1+R)^N} \sum_{i=0}^{N} \binom{N}{i} q^i \, (1-q)^{N-i} \, H(N, i).$$

**Example (one period):** With the default values $S_0 = K = 100$, $U = 0.05$, $D = -0.05$, $R = 0.01$, and $N = 1$, the stock price goes up to 105 or down to 95, so the option pays 5 or 0. The risk-neutral probability is $q = 0.06/0.10 = 0.6$, and the price of the option is

$$H(0, 0) = \frac{0.6 \cdot 5 + 0.4 \cdot 0}{1.01} \approx 2.9703.$$

# Part 1: Setting up the project

**Create a private repository** on QMUL GitHub and name it `binomial_pricer`. **Clone it** to the Jupyter Hub (or your own computer), as in Lecture 6. Then **set up the project structure**: a `CMakeLists.txt` file, a `.gitignore` file that excludes the `build` directory and all compiled files, and a `README.md` file. A possible layout is:

```
binomial_pricer/
├── CMakeLists.txt
├── README.md
├── .gitignore
├── include/
│   ├── binomial.hpp
│   └── menu.hpp
├── src/
│   ├── binomial.cpp
│   ├── menu.cpp
│   └── main.cpp
└── analysis/
```

Here, `binomial.hpp` and `binomial.cpp` contain the model (the actual pricing), whereas `menu.hpp` and `menu.cpp` contain the functions for the user interface (reading and printing). The `analysis` directory is a good place for your data, plotting scripts, and figures. You are free to organize your files differently, but **keep the model and the user interface in separate files**, and **put all your functions into a namespace** (e.g., `pricer`), **except for `main()`**, which must not be in any namespace for the program to be valid C++.

Base your `CMakeLists.txt` on the example from Lecture 6, which enables all compiler warnings (`-Wall`) for Debug builds, and **configure your build directory as a Debug build** with

```
cmake -DCMAKE_BUILD_TYPE=Debug ..
```

This way, the compiler warns you about potential problems while you develop your program. Make your **first commit** and **push** it as soon as the skeleton builds.

**Use git throughout the project.** Commit small, self-contained changes with meaningful commit messages (e.g., “Add backward induction pricing” rather than “update”), and push at the end of every work session. Don’t worry if you realize later that a commit contained a bug; that’s perfectly normal. Just fix it and commit again. As a guideline, aim for **at least 5 commits on at least 3 different days**. A clean history with good messages is more valuable than a large number of commits. Never commit compiled files or executables.

# Part 2: The pricing model

**Implement the binomial model** in `binomial.hpp` and `binomial.cpp`. How you split the work into functions is up to you, but here is a suggestion:

```
bool isArbitrageFree(double up, double down, double rate);
double riskNeutralProbability(double up, double down,
        double rate);
double stockPrice(double spot, double up, double down, int
        n, int i);
double callPayoff(double spot, double strike);
double priceEuropeanCall(double spot, double strike, double
        up,
                         double down, double rate, int
        periods);
```

The function `priceEuropeanCall` should compute the price by **backward induction** as introduced in the Background section above.

**Test your functions** before you build the user interface, e.g., with hard-coded parameter values in `main`. Here are some ways to check that your program is correct:

- Compare with the one-period example above, and work out a two-period example by hand.
- Compare with the following reference values for $S_0 = K = 100$, $U = 0.05$, $D = -0.05$, $R = 0.01$:

| $N$ | 1 | 2 | 10 | 50 |
|---|---|---|---|---|
| Price | 2.9703 | 3.6173 | 11.8186 | 40.1445 |

- Implement the explicit formula for the price as a second function and compare both results for various parameters.
- Check limiting cases: What should the price be for $K = 0$? For a very large $K$? The price should always lie between $\max\left(S_0 - K/(1+R)^N, 0\right)$ and $S_0$.

**Summarize in your report** how you convinced yourself that your program is correct.

# Part 3: The user interface

**Write a menu-based user interface.** When the program starts, all parameters should be set to their default values (see the table above). The program then displays a menu in a loop. The menu lists all parameters with an index and their current value, followed by the available actions. It could look like this, for example:

```
Current parameters:
  [1] Current price S0      : 100.0000
  [2] Strike price K        : 100.0000
  [3] Up rate U             :   0.0500
  [4] Down rate D           :  -0.0500
  [5] Interest rate R       :   0.0100
  [6] Number of periods N   :       10
Actions:
  [7] Compute option price
  [0] Quit
Select an option:
```

If the user selects a parameter, the program asks for a new value for that parameter and then shows the menu again. If the user selects “Compute option price”, the program prints the price and the risk-neutral probability, e.g.,

```
Fair price of the European call: 11.8186 (risk-neutral
probability q = 0.6000)
```

Feel free to add further actions if they are useful to you, e.g., for your experiments in Part 5.

Your program should **handle invalid input gracefully**:

- **Reject invalid input** with an informative message and ask again: non-numeric input, menu choices that don’t exist, and parameter values that violate the constraints in the table above (e.g., a negative stock price or $N = 0$). You may also introduce a sensible upper bound for $N$.
- **Check the no-arbitrage condition $D < R < U$ before computing the price.** If it is violated, explain the problem to the user and don’t print a price. Note that you should *not* reject a new value for $U$, $D$, or $R$ just because it violates the condition: When the user changes the parameters one at a time, the condition may be violated temporarily.
- **End the program cleanly** if the end of the input is reached (Ctrl-D).

**How to store the parameters?** Since we haven’t covered any way to group several variables into one object yet, you need to make a design decision. Two possible approaches are:

- **(a)** six separate variables, and an `if`/`else if` chain to handle the menu selection;
- **(b)** two parallel vectors, one with the names and one with the values of the parameters, indexed by the menu number.

Both have advantages and disadvantages. Choose one (or come up with your own) and **justify your choice briefly in your report**.

# Part 4: Documentation

**Document every function** in your header files with a Doxygen comment (`/** ... */`) including `@param` and `@return` tags, as discussed in Lecture 5.

**Write a `README.md`** that explains what your program does, how to build it with CMake, and how to use it (e.g., with an example session). Another person should be able to build and run your program using only your README. In Week 8, someone will do exactly that!

# Part 5: Experiments and report

**Use your program to explore how the option price depends on the parameters.** You decide which questions to investigate. Some ideas:

- How does the price depend on the strike $K$?
- How does it depend on the interest rate $R$?
- How does it depend on the spread $U - D$ between up and down rates? (This plays the role of the volatility of the stock.)
- How does it depend on the number of periods $N$ if the other parameters are kept fixed?
- What happens close to the boundary of the no-arbitrage condition, e.g., as $R$ approaches $D$ or $U$? How can you interpret $q$ in these cases?

**Producing data for plots:** As in Problem 6 of Problem Set 5, you can print your results in the CSV format and redirect the output into a file. Since your menu also prints to the standard output, you may find it convenient to write a **second small program** for your experiments that uses the same model files, but no menu: A second `add_executable(...)` line in your `CMakeLists.txt` that lists another `main` file (e.g., `src/sweep.cpp`) together with `src/binomial.cpp` is all it takes. Separating the model from the user interface pays off here. You can plot your results with any tool you like (e.g., Python with matplotlib, as in Problem Set 5).

**Write a report of about two pages** (PDF) that covers:

1. **Results:** Present the results of your experiments in tables and/or figures, with axis labels and captions, and discuss them.
2. **Choice of parameters:** Explain *why* you chose these particular parameter values. For instance, what length of a period do they correspond to? Are they realistic? Which effect did you want to isolate?
3. **Interpretation:** Do the trends you observe make sense from a financial point of view? Did anything surprise you?
4. **Program design and testing:** Briefly justify how you stored the parameters (Part 3) and any other design choices you made, and summarize how you checked that your program is correct (Part 2).
5. **Reflection:**
    - How did your work progress over the two weeks? Refer to your git history.
    - What went well, and what was difficult?
    - Which feedback that you received in class, from the lecturer, and from your peers in Weeks 1 to 5 did you take into account in this project, and how?
    - Did you use AI tools? If so, how did you scrutinize their output and what changes did you have to make?

The report doesn’t need to repeat the background on the binomial model from this sheet.

# Submission

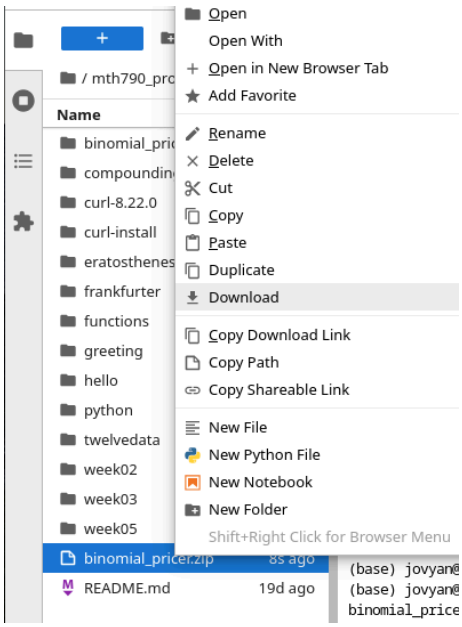
Before the deadline (**Friday, 6 November 2026, 23:59 UK time**), **commit and push your final version** to QMUL GitHub and submit the following two files on **QMPlus** to the corresponding submission points:

1. **`report.pdf`:** your report. Don't add it to your git repository.
2. **A zip archive of your complete project folder**, including the hidden `.git` directory, which contains your git history. Please leave out the `build` directory.

On the Jupyter Hub, you can create the zip archive by opening a terminal, changing to the **parent directory** of your project folder, and running

```
zip -r binomial_pricer.zip binomial_pricer -x
"binomial_pricer/build/*"
```

(adapt the folder name if yours is different). You can then download the zip archive by selecting and right-clicking it in the Jupyter Hub and then clicking "Download":




Alternatively, clone your repository to your own computer (even if you wrote all code on the Jupyter Hub) and create the zip archive from your local copy with a graphical tool (Explorer, Finder, …). In this case, make sure that hidden files and folders are included, but the `build` directory is excluded.

To **check** that your zip archive contains the git history, run

```
unzip -l binomial_pricer.zip | grep ".git/HEAD"
```

which should print one line. On Windows, these commands are usually not available. Instead, open the zip archive in Explorer (with hidden items shown) and check that it contains the `.git` folder of your project.

# Checklist

Use this checklist before you submit. The peer review in Week 8 and my written feedback will follow the same points.

- [ ] The program builds with CMake following the instructions in the README, without warnings in a Debug build (with `-Wall`, see Part 1).
- [ ] The computed prices agree with the reference values, and the no-arbitrage condition is checked before pricing.
- [ ] The menu shows all parameters; invalid input and Ctrl-D are handled gracefully.
- [ ] The model and the user interface are in separate header and source files, and all functions are in a namespace.
- [ ] The code uses only concepts from Lectures 1–6.
- [ ] Every function in the header files has a Doxygen comment, and the README is complete.
- [ ] The git history shows regular commits with meaningful messages, there are no compiled files in the repository, and the final version is pushed to QMUL GitHub.
- [ ] The report presents and interprets results, justifies the choice of parameters, and reflects on your progress and the feedback you received, in about two pages.

# Feedback

In **Week 8**, you will be assigned another student's project. You will receive their report and zipped project folder from me, so that you work with exactly the submitted version. **Build and test their program** (and try to break it!), **read their report**, and **give them constructive feedback** along the points of the checklist above. You will upload your feedback on QMPlus. Afterwards, I will review the projects together with the peer feedback, and you will receive individual written feedback from me.